minicut Video Editor

Embedded LuaJIT & Plugin API Reference Manual

LuaJIT 5.1 Runtime MEL-Style Panels Maya Shelf Dock Built-in Network (libcurl)

1. Introduction & Architecture

minicut features a fully embedded, high-performance **LuaJIT** scripting engine. Similar to Maya's MEL and Python architecture, minicut enables automation, timeline editing, clip keyframing, and custom interactive dockable tool panels.

Script Directory Locations

Scripts and icons are discovered in the user's home configuration directory:

Location	Trigger / Role	Description
`~/.config/minicut/autorun/`	Startup / Auto-register	Scripts execute on startup and generate an icon button on the horizontal Maya Shelf Dock below the main menu bar.
`~/.config/minicut/scripts/`	Menu Click / On Demand	Scripts appear in the top-level **Scripts** menu on the main menu bar. Clicking runs the script and opens its tool panel.
`~/.config/`	Shelf Icon Asset	32x32 PNG icon images matching script stems (e.g.,

Location	Trigger / Role	Description
`minicut/icons/`		`quick_placement.png` for `quick_placement.lua`).

> **Maya Shelf Fallback Badges**
>
> If an autorun script does not have a corresponding PNG icon in `~/.config/minicut/icons/`, minicut automatically creates a sleek 2-letter uppercase badge (e.g., `QP` for Quick Placement) on the shelf button.

2. MEL-Style Script Architecture

Every minicut script follows an intuitive, modular structure. A script defines timeline and clip manipulation callbacks, followed by a `plugin:init_panel()` method that registers Dear ImGui UI controls.

```
-- ============================================================================

-- Example Base Script: my_tool.lua

-- ============================================================================



local target_track = 0

local clip_scale = 1.25



-- Timeline and clip manipulation logic

function execute_placement()

    minicut.push_undo()

    local path = "/path/to/media.mp4"

    local clip_id = timeline.place_clip(path, target_track, timeline.get_playhead(),
5.0)

    if clip_id then
```

```
        clip.set_scale(clip_id, clip_scale)

        minicut.set_status("Placed media clip on track " .. target_track)

    end

end



-- GUI panel declaration

function plugin:init_panel()

    gui.text("Media Placement Tool")

    gui.separator()

    target_track = gui.slider_int("Track Lane", target_track, 0, 4)

    clip_scale = gui.slider_float("Initial Scale", clip_scale, 0.1, 5.0)



    if gui.button("Place Clip At Playhead", execute_placement) then

        -- Optional inline handling

    end

end
```

When executed, each script automatically opens an independent, dockable tool panel situated to the right of the preview and timeline panels.

3. Core Lua Modules

3.1. Project & File Automation (`project` / Global)

Function	Returns	Description
`project_save([path])` `project.save([path])`	`boolean, string`	Saves current project state to a `.minicut` JSON file. If path is omitted, saves to the current project

Function	Returns	Description
		file path.
`project_load(path)` `project.load(path)`	`boolean, string`	Loads a project from `path`, updates timeline, resets playhead, and refreshes the audio engine and video preview.
`project.get_name()`	`string`	Returns the current project name.
`project.set_name(name)`	`void`	Sets the current project name.
`project.import_video(path)`	`string (asset_id)`	Imports a video file into the project's media asset pool.
`project.import_audio(path)`	`string (asset_id)`	Imports an audio file into the project's asset pool.
`project.import_image(path, [dur])`	`string (asset_id)`	Imports a static image file into the project's asset pool (default duration: 5.0s).

3.2. Network & Internet Library (`net`)

Powered by embedded **libcurl**, the `net` module enables downloading remote media, uploading exports to web servers, and calling REST APIs.

Function	Returns	Description
`net.get(url, [opts])`	`table	(nil, err)`	Sends an HTTP GET request. Returns table: `{ status, ok, body, headers }`.
`net.post(url, body, [opts])`	`table	(nil, err)`	Sends an HTTP POST request with string or key-value table body.
`net.download(url, dest, [opts])`	`boolean, string`	Streams remote file directly to disk at `dest` without RAM overhead.
`net.upload(url, file, [opts])`	`table	(nil, err)`	Uploads local file via multipart/form-data. Supports

Function	Returns	Description
		custom `field_name` and `extra_fields`.
`net.request(opts)`	`table | (nil, err)`	Generic HTTP request with custom method (GET, POST, PUT, DELETE, PATCH).

```
-- Example: Downloading stock music and importing directly to timeline

local ok, path = net.download("https://example.com/soundtrack.mp3",
"/tmp/soundtrack.mp3")

if ok then

    local clip_id = timeline.place_clip(path, 1, 0.0, 30.0)

    minicut.set_status("Downloaded and placed soundtrack!")

end
```

3.3. GUI Controls (`gui`)

Function	Returns	Description
`gui.button(label, [callback])`	`boolean`	Standard push button. Invokes `callback` when clicked.
`gui.small_button(label, [cb])`	`boolean`	Compact button without vertical frame padding.
`gui.text(text)`	`void`	Standard text label.
`gui.text_colored(text, r, g, b, [a])`	`void`	Colored text label (color components 0.0 to 1.0).
`gui.text_disabled(text)`	`void`	Muted gray text label.
`gui.separator()`	`void`	Horizontal dividing line.
`gui.same_line([offset], [spacing])`	`void`	Places the next control on the same

Function	Returns	Description
		horizontal line.
`gui.slider_float(lbl, val, min, max)`	`number`	Interactive floating-point horizontal slider.
`gui.slider_int(lbl, val, min, max)`	`integer`	Interactive integer horizontal slider.
`gui.drag_float(lbl, val, [spd, min, max])`	`number`	Click-and-drag floating-point number field.
`gui.checkbox(lbl, val)`	`boolean`	Toggle checkbox.
`gui.input_text(lbl, val)`	`string`	Single-line text input field.
`gui.progress_bar(fraction, [text])`	`void`	Progress bar (0.0 to 1.0).
`gui.tooltip(text)`	`void`	Displays hover tooltip over preceding item.

3.4. Timeline & Tracks (`timeline`)

Function	Returns	Description
`timeline.place_clip(path, track, t, dur)`	`string (clip_id)`	Imports media and places clip onto track lane at timestamp `t`.
`timeline.get_playhead()`	`number`	Returns current playhead time in seconds.
`timeline.set_playhead(time_sec)`	`void`	Seeks playhead to specified time and updates video/audio preview.
`timeline.get_duration()`	`number`	Returns total project duration in seconds.
`timeline.get_track_count()`	`integer`	Returns total number of timeline tracks.
`timeline.get_track_depth(track_i`	`integer`	Returns Z-depth compositing order for

Function	Returns	Description
`dx)`		track (0=back, 100=front).
`timeline.set_track_depth(track_idx, z)`	`void`	Sets Z-depth compositing order for track.
`timeline.add_track(track_type)`	`string (id)`	Appends a new track (`"video"` or `"audio"`).
`timeline.delete_clip(clip_id)`	`boolean`	Deletes clip from timeline.
`timeline.split_clip(clip_id, split_t)`	`boolean`	Splits clip into two halves at timestamp `split_t`.

3.5. Clip Transforms & Keyframes (`clip` & `timeline`)

Function	Returns	Description
`clip.get_selected_id()`	`string`	Returns ID of currently selected clip.
`clip.get_position(clip_id)`	`x, y`	Returns clip pixel translation offsets.
`clip.set_position(clip_id, x, y)`	`void`	Sets clip translation offsets.
`clip.get_scale(clip_id)`	`number`	Returns scale multiplier (default 1.0).
`clip.set_scale(clip_id, scale)`	`void`	Sets clip scale multiplier.
`clip.get_rotation(clip_id)`	`number`	Returns rotation angle in degrees.
`clip.set_rotation(clip_id, deg)`	`void`	Sets rotation angle in degrees.
`clip.get_opacity(clip_id)`	`number`	Returns opacity (0.0=invisible, 1.0=solid).
`clip.set_opacity(clip_id, alpha)`	`void`	Sets clip opacity.
`clip.get_volume(clip_id)`	`numbe`	Returns audio gain (1.0 = 100%).

Function	Returns	Description
	`r`	
`clip.set_volume(clip_id, vol)`	`void`	Sets audio gain.
`timeline.add_keyframe(id, prop, t, val)`	`void`	Adds animation keyframe (prop: `"pos_x"`, `"pos_y"`, `"scale"`, `"rotation"`, `"opacity"`, `"volume"`).
`timeline.evaluate_keyframe(id, prop, t, def)`	`numbe r`	Evaluates interpolated value at timestamp `t`.

3.6. Engine Utilities (`minicut`)

Function	Returns	Description
`minicut.push_undo()`	`void`	Pushes the current project snapshot onto the Undo stack.
`minicut.set_status(msg, [dur])`	`void`	Displays green notification toast in the main menu bar.
`minicut.log(message)`	`void`	Logs message to the console standard output.

4. Comprehensive Plugin Examples

Example 1: Keyframe Bouncing Logo Animator

```
-- ~/.config/minicut/autorun/logo_bounce.lua

local bounce_dur = 2.0

local peak_height = -150.0



function apply_bounce_animation()
```

```
    local cid = clip.get_selected_id()
    if cid == "" then
        minicut.set_status("Please select a clip first!")
        return
    end

    minicut.push_undo()
    local cur_t = timeline.get_playhead()
    timeline.clear_keyframes(cid, "pos_y")

    -- Animate bounce cycle
    timeline.add_keyframe(cid, "pos_y", cur_t, 0.0)
    timeline.add_keyframe(cid, "pos_y", cur_t + (bounce_dur * 0.5), peak_height)
    timeline.add_keyframe(cid, "pos_y", cur_t + bounce_dur, 0.0)

    minicut.set_status("Applied bounce keyframes to clip!")
end

function plugin:init_panel()
    gui.text("Bouncing Logo Animator")
    gui.separator()
    bounce_dur = gui.slider_float("Cycle Duration (s)", bounce_dur, 0.2, 5.0)
    peak_height = gui.slider_float("Peak Jump (px)", peak_height, -400.0, 0.0)
    gui.button("Apply Bounce Animation", apply_bounce_animation)
```

```
end
```

Example 2: Automated Cloud Project Backup

```
-- ~/.config/minicut/scripts/cloud_backup.lua

local upload_endpoint = "https://httpbin.org/post"



function execute_backup()

    local temp_file = "/tmp/minicut_auto_backup.minicut"

    local save_ok, err = project_save(temp_file)

    if not save_ok then

        minicut.set_status("Failed to save local snapshot: " .. tostring(err))

        return

    end



    minicut.set_status("Uploading project backup...")

    local resp = net.upload(upload_endpoint, temp_file, {

        field_name = "project_file",

        extra_fields = {

            timestamp = tostring(os.time()),

            project_name = project.get_name()

        }

    })



    if resp and resp.ok then

        minicut.set_status("Project successfully backed up to cloud!")
```

```
    else

        minicut.set_status("Backup upload failed: " .. tostring(resp and resp.status or
"network error"))

    end

end



function plugin:init_panel()

    gui.text("Cloud Project Backup Tool")

    gui.separator()

    upload_endpoint = gui.input_text("Endpoint URL", upload_endpoint)

    gui.button("Backup Project Now", execute_backup)

end
```